Lec 13 & 14 - Observer, Singleton, Model-View-Controller

COMPSCI 220 - Perretta

Overview for the Week

Main goal: **Think critically about how to structure large software applications**

Sub-topics:

- **YAGNI** principle ("You Aren't Going to Need It")
- **Observer Pattern**: *Decouples* data source from entities that consume the data
- **Singleton Pattern**: Managing global application state
- **Model-View-Controller**: Common architecture for Graphical User Interface (GUI) applications

I recommend reading the lecture notes for this week

Design Patterns

Describe **well-known solutions** to **specific problems**.

- How do parts of a system **interact**?
- What are the **roles** of each part?

"Interface" vs `interface`

- `interface` is a language feature
- "Interface" is higher level of abstraction
  - e.g., `interface` vs. `class` is **implementation detail** of design pattern

Recommended Reading

"Gang of Four" Design Patterns book:
https://en.wikipedia.org/wiki/Design_Patterns

Lecture notes from a class I took:
https://websites.umich.edu/~eecs381/lecture/notes.html

Notes & historical docs from a creator of MVC:
https://folk.universitetetioslo.no/trygver/themes/mvc/mvc-index.html

The YAGNI Principle

"You Aren't Going to Need It"

What does "extensibility" mean to you?

- No design is infinitely extensible
  - Impossible to anticipate & design for all possible future requirements
- Over-designing can lead to worse design

Design patterns describe solutions to **specific problems.**

If you have a different problem, **don't use that pattern!**

The Observer Pattern

Motivating Example

What's wrong with this code? (Live web page example)

```
// Run the arrow function every N milliseconds
setInterval(() => {
  // Read the value from the HTML input
  const value = document.getElementById("name-input").value;

  // Show the input text elsewhere on the page
  document
    .getElementById("name-display")
    .replaceChildren(document.createTextNode(value));
}, 1000);
```

(Full code example in lecture notes)

Motivating Example

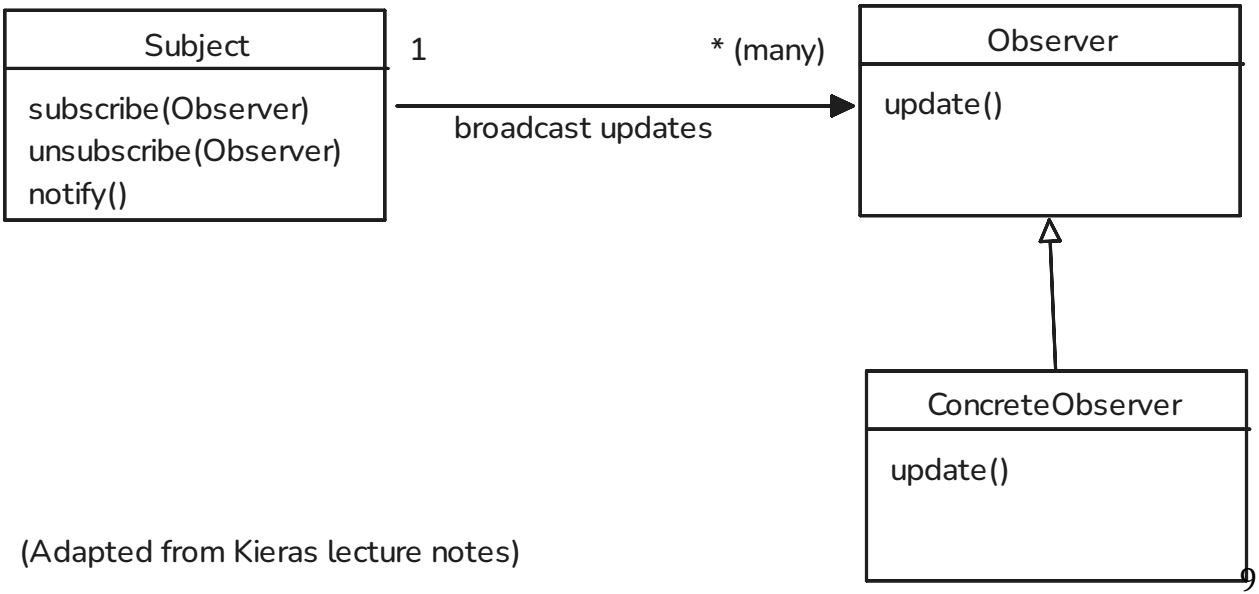
A better solution (Live web page example)

```
// Arrow func called the user types input
document.getElementById("name-input").addEventListener("keyup", (event) => {
  // Read the value from the input
  const value = document.getElementById("name-input").value;

  // Show the input text elsewhere on the page
  document
    .getElementById("name-display")
    .replaceChildren(document.createTextNode(value));
});
```

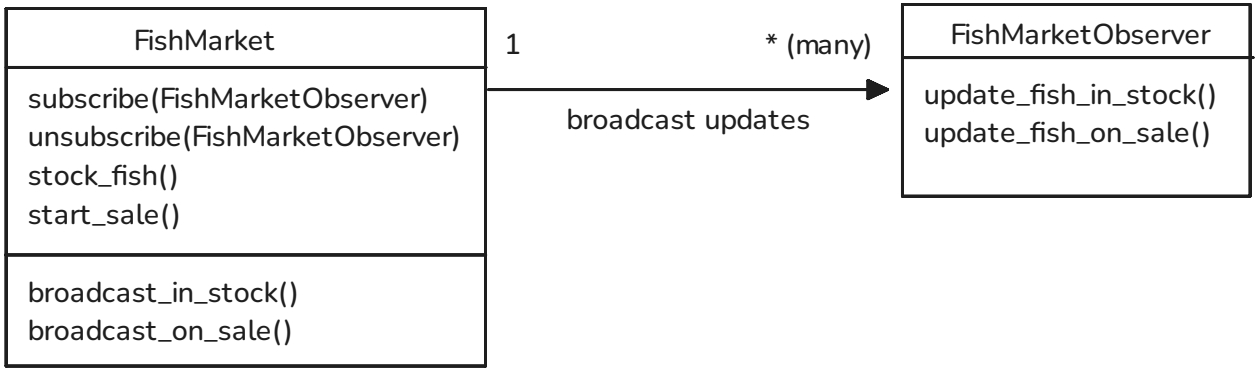
(Full code example in lecture notes)

Observer Pattern

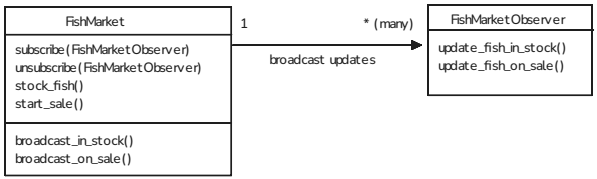


(Adapted from Kieras lecture notes)

Concrete Example: Fish Market



Concrete Example: Fish Market



- Observers ignore updates they don’t need
  - Is that inefficient?
- Should we combine into one "update" function?

"Push" vs. "Pull"

Push:

- Update has full info
  - E.g., "Swordfish is in stock"

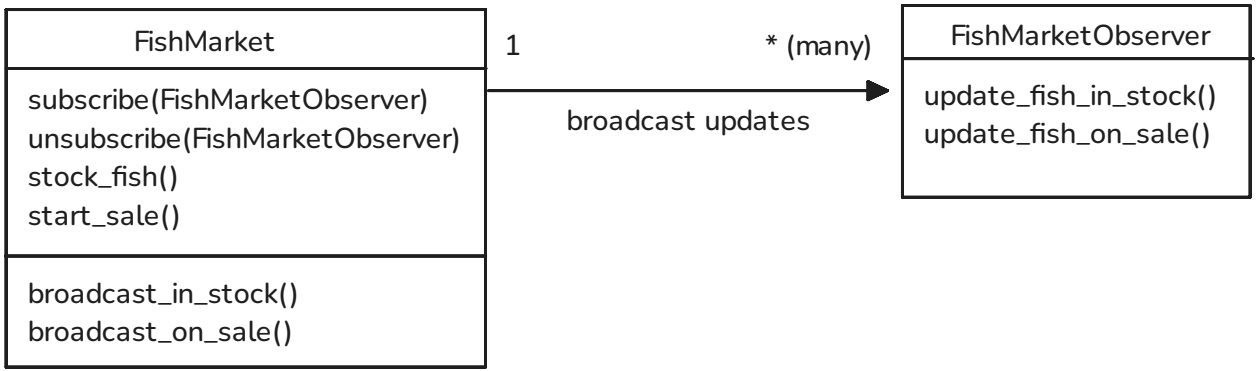
Pull:

- Update only says something changed, not what
  - E.g., "New fish in stock"

Is one strategy better than the other? For the fish market example? In general?

Implementing the Fish Market

(Live coded in class, see lecture notes for full code)



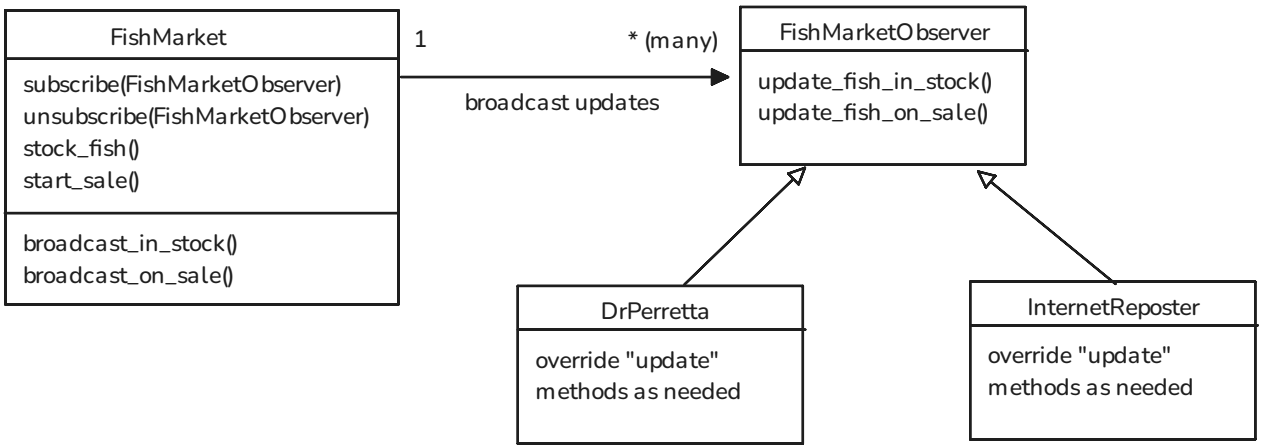
Key Benefits of Decoupling

Because the Subject is **decoupled** from the Observers, we **successfully implemented the entire Subject *before adding any Observers*.**

We can add Observers now **without changing the subject**.

Adding Concrete Observers

(Live coded in class, see lecture notes for full code)



What's Wrong with this Design?

```
class FishMarket {
  constructor(
    private dr_perretta: DrPerretta2,
    private internet_reposters: InternetReposter2[],
  ) {}

  stock_fish(fish: string) {
    if (fish === "swordfish") {
      this.dr_perretta.buy_swordfish();
    }
  }

  start_sale(fish: string) {
    for (const reposter of this.internet_reposters) {
      reposter.post_fish_on_sale(fish);
    }
  }
}
```

```
class InternetReposter2 {
  post_fish_on_sale(fish: string) {
    // omitted for brevity
  }
}

class DrPerretta2 {
  buy_swordfish() {
    // omitted for brevity
  }
}
```

`interface` vs. Abstract Base Class?

For the Observer "Interface", which should we use?

Should we add…

Should the subject send notifications for when a fish is out of stock? I feel like some observers might need that.

You Aren’t Going to Need It

Design for what you **know you need**, not what you **think you might need**.

The Singleton Pattern

Global Variables?

What can you tell me about global variables?

Global State

Many large applications have global state.

We want **controlled access** to that global state.

Global variables could be:

- changed any time
- changed from anywhere
- changed in any way

Singleton Pattern

1. Guarantee only **one instance** of a class
2. Make that instance **globally accessible**
3. Provide **controlled access** to state

Singleton Pattern

1. Guarantee only **one instance** of a class
2. Make that instance **globally accessible**
3. Provide **controlled access** to state

1. `private` constructor
2. `static` `getInstance` method
3. Methods/member vars for state

Implementing a Singleton

(Live coded in class, see notes for starter code example)

Interacting with the Singleton Class

(Live coded in class, see notes for starter code example)

```
MyAppData.getInstance().addThing("walugi", 42);
MyAppData.getInstance().addThing("sanic", 43);
console.log(MyAppData.getInstance().findThing("sanic"));
MyAppData.getInstance().duplicateThing("waluigi");
```

Controlled access to application state.

Do I have to call `getInstance()` every time??

Yes.

Seriously??

This might be tempting, but **DON'T DO IT**

```
// DON'T DO THIS
const data = MyAppData.getInstance();
data.duplicateThing("waluigi");
```

Why??

- `getInstance()` guarantees in instance will be available **when called**
- How the instance is managed is an **implementation detail**

(See notes for an example, can discuss in class if time)

Why not another approach?

Other approaches exist (e.g., "dependency injection")

Singleton is relatively simple to implement, good for our purposes

Recap

The **Observer Pattern** lets us **decouple** a data source from consumers of the data.

The **Singleton Pattern** gives us **controlled access** to **global state**.

Design for your **current need**. **Don't over-design** for hypothetical future needs.

Next time: Model-View-Controller

What's Wrong with this Design?

```
class FishMarket {
  constructor(
    private dr_perretta: DrPerretta2,
    private internet_reposters: InternetReposter2[],
  ) {}

  stock_fish(fish: string) {
    if (fish === "swordfish") {
      this.dr_perretta.buy_swordfish();
    }
  }

  start_sale(fish: string) {
    for (const reposter of this.internet_reposters) {
      reposter.post_fish_on_sale(fish);
    }
  }
}
```

```
class InternetReposter2 {
  post_fish_on_sale(fish: string) {
    // omitted for brevity
  }
}

class DrPerretta2 {
  buy_swordfish() {
    // omitted for brevity
  }
}
```

Mystery from Last Time

Inside Singleton constructor:

```
private constructor() {
  Singleton.instance = new Singleton();
}
```

Type checker didn’t produce an error!

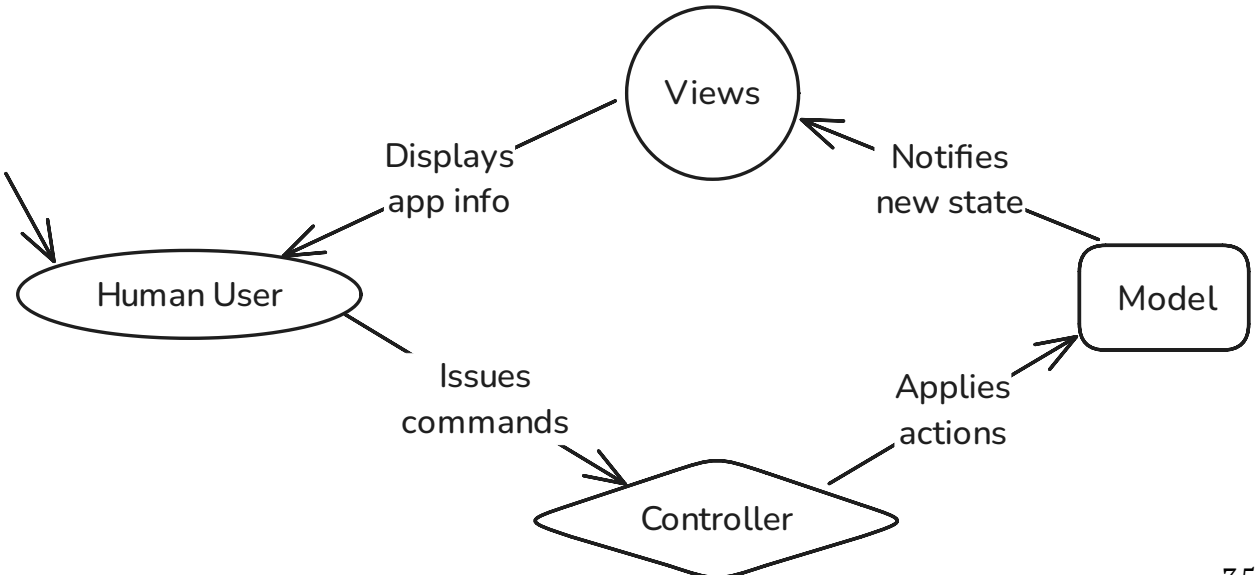
Running the code solved the mystery.

(It causes infinite recursion)

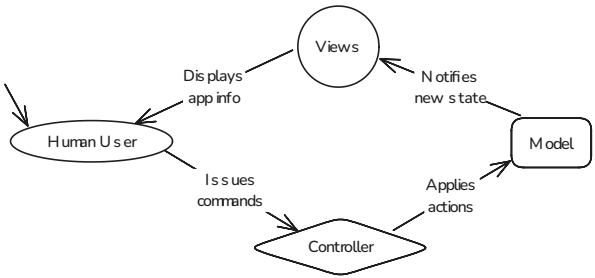
Model-View-Controller

Lec 12 - COMPSCI 220 - Perretta

Model-View-Controller (MVC)



MVC Responsibilities: **Model**

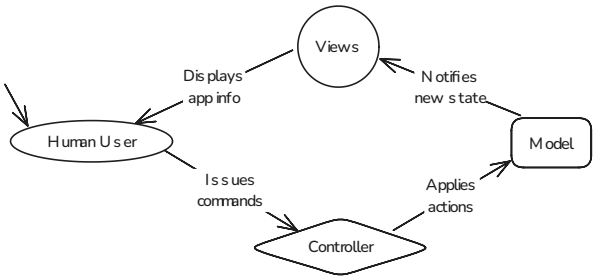


Encapsulates **core data & logic** of application.

Many options how to implement, we'll use singleton.

Brainstorm: "Model" in other applications?

MVC Responsibilities: **View**



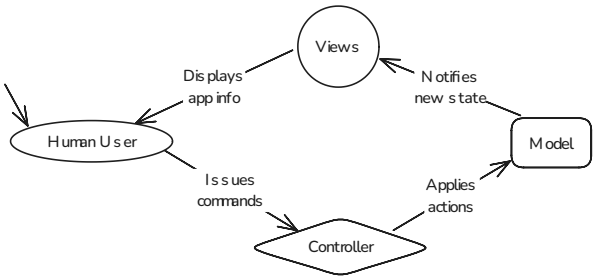
Displays application state to the **user**.

Typically multiple views, multiple open at once.

Receives updates from the Model (sound familiar?)

Does **not** apply user commands to the model.

MVC Responsibilities: **Controller**



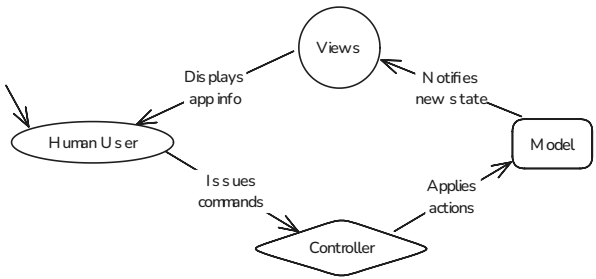
Processes commands from the user & **manages views**.

User action -> Controller calls Model methods.

Opens Views & attaches them to Model.

Closes Views & detaches them from Model.

MVC Responsibilities: Interactions

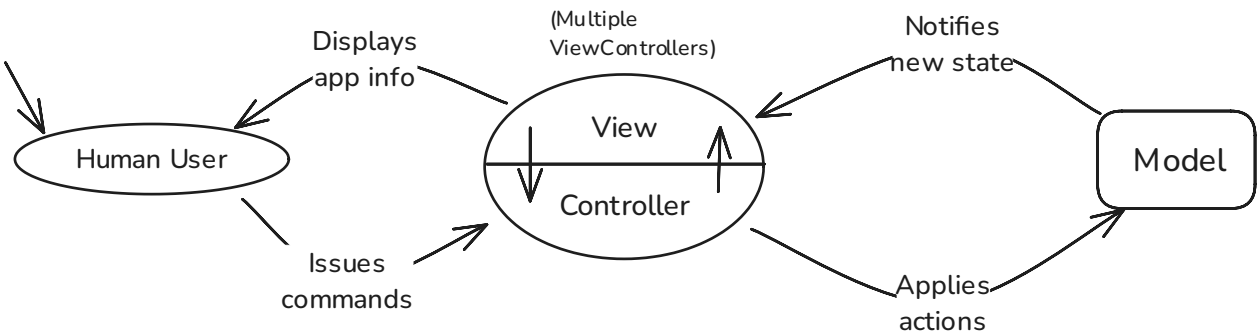


How much does the _____ know about the _____?

- Model, View
- Model, Controller
- View, Model
- View, Controller
- Controller, Model
- Controller, View

(Answer in terms of coupling and interaction with derived classes)

Variation: ViewController in GUI Applications

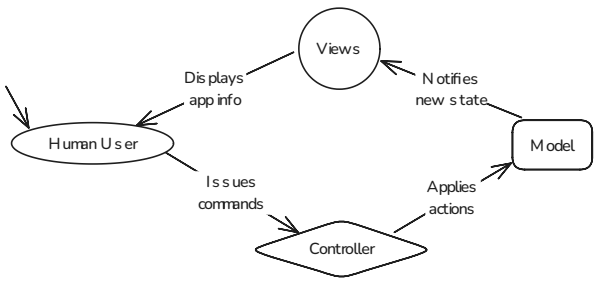


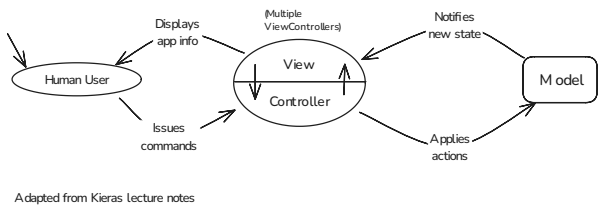
Adapted from Kieras lecture notes

View and Controller combined as ViewController.

Recap

MVC **decouples** the core **application state/logic** from the **user interface**



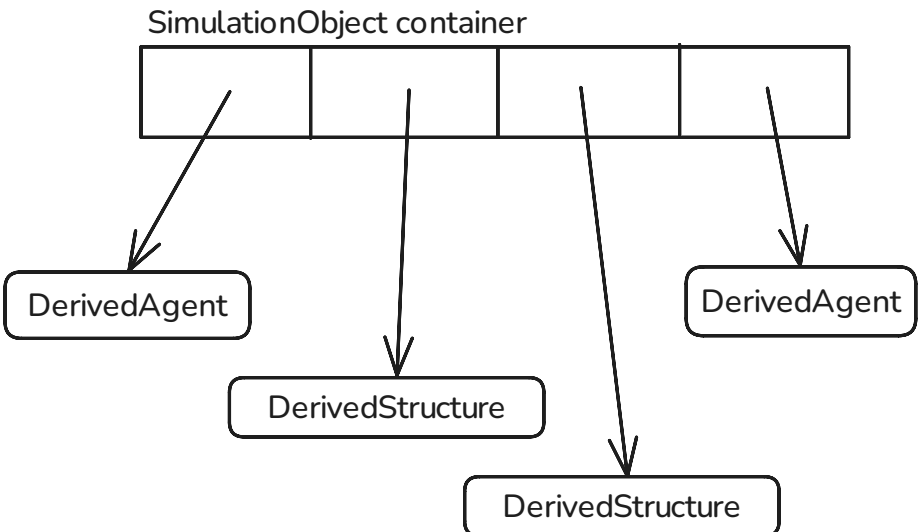


Adapted from Kieras lecture notes

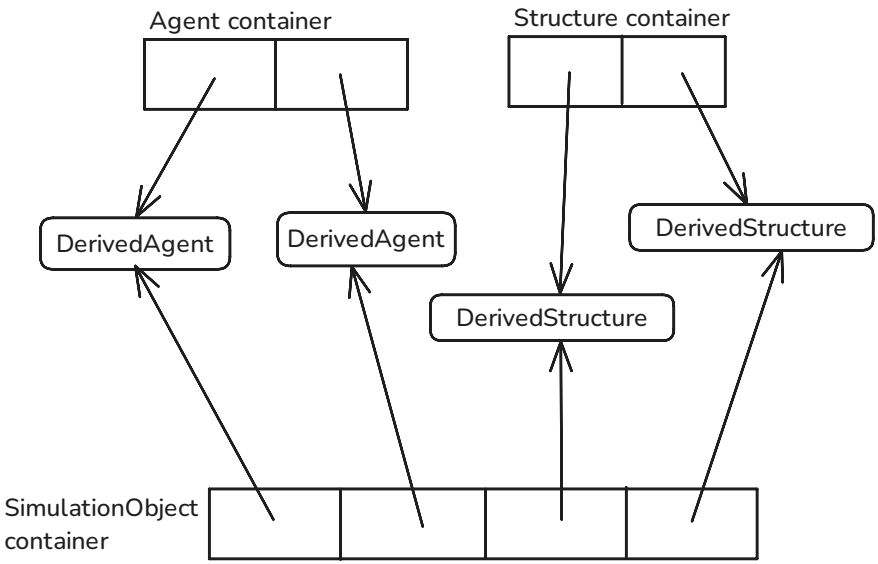
Tried & true strategy for making this kind of system **extensible**

Extra: "Separate Containers"

What if we need to interact with a specific type of object in this container?



Extra: "Separate Containers"



Extra: Nominal vs. Structural Subtyping

What would this code do in Java?

```
abstract class MarioBrother {
  abstract speak(): void;
}

class Luigi extends MarioBrother {
  speak() {
    console.log("Luiiiigi!");
  }
}

class Waluigi {
  speak() {
    console.log("WAAAAAAAAALUIGI TIME!");
  }
}
```

```
function startGame(character: MarioBrother) {
  character.speak();
}

startGame(new Luigi());
startGame(new Waluigi());
```

Extra: Nominal vs. Structural Subtyping

Do we get an error in typescript?

```
abstract class MarioBrother {
  abstract speak(): void;
}

class Luigi extends MarioBrother {
  speak() {
    console.log("Luiiiigi!");
  }
}

class Waluigi {
  speak() {
    console.log("WAAAAAAAAAALUIGI TIME!");
  }
}
```

```
function startGame(character: MarioBrother) {
  character.speak();
}

startGame(new Luigi());
startGame(new Waluigi());
```

```
$ tsc && node out/structural_subtyping.js
Luiiiigi!
WAAAAAAAAAALUIGI TIME!
```

TypeScript Uses **Structural Subtyping**

```
abstract class MarioBrother {
  abstract speak(): void;
}

class Luigi extends MarioBrother {
  speak() {
    console.log("Luiiiigi!");
  }
}

class Waluigi {
  speak() {
    console.log("WAAAAAAAAAALUIGI TIME!");
  }
}
```

- The name of the class **does not matter**
- Checks if the **attributes & methods** are compatible.

TypeScript Uses **Structural Subtyping**

```
abstract class MarioBrother {
  abstract speak(): void;
}

class Waluigi {
  speak() {
    console.log("WAAAAAAAAAALUIGI TIME!");
  }

  squeegee() {
    console.log("Walusqueeeeeegeeee!")
  }
}

function startGame(character: MarioBrother) {
  character.speak();
}

startGame(new Waluigi());
```

```
WAAAAAAAAAALUIGI TIME!
```

TypeScript Uses **Structural Subtyping**

```
abstract class MarioBrother {
  abstract speak(): void;
}

class Waluigi {
  speak(numWaas: number) {
    for (let i = 0; i < numWaas; ++i) {
      console.log("WAAAAAAAAAALUIGI TIME!");
    }
  }
}

function startGame(character: MarioBrother) {
  character.speak();
}

startGame(new Waluigi());
```